Exercise 5: Tuning and Final Evaluation – SOLUTIONS

Machine Learning with R in the Life Sciences

Setup

```r
library(tidyverse)
library(tidymodels)
library(vip)

set.seed(2025)
n_samples <- 200; n_genes <- 50; n_de_genes <- 10
sample_class <- factor(rep(c("tumor", "normal"), each = n_samples / 2))
expression_matrix <- matrix(rnorm(n_samples * n_genes, mean = 8, sd = 1.5),
                            nrow = n_samples, ncol = n_genes)
colnames(expression_matrix) <- paste0("gene_", 1:n_genes)
for (i in 1:n_de_genes) {
  expression_matrix[1:(n_samples / 2), i] <-
    expression_matrix[1:(n_samples / 2), i] + rnorm(n_samples / 2, mean = 2, sd = 0.5)
}
gene_data <- as.data.frame(expression_matrix) |> mutate(class = sample_class)

set.seed(123)
split <- initial_split(gene_data, prop = 0.75, strata = class)
train_data <- training(split)
test_data  <- testing(split)

set.seed(42)
cv_folds <- vfold_cv(train_data, v = 5, strata = class)
```

5.1

```r
rf_tunable <- rand_forest(mtry = tune(), trees = tune(), min_n = tune()) |>
  set_mode("classification") |>
  set_engine("ranger", importance = "impurity")
```

5.2

```r
rf_grid <- grid_regular(
  mtry(range = c(3, 20)),
  trees(range = c(200, 800)),
  min_n(range = c(2, 10)),
  levels = 3
```

```r
)
cat("Combinations:", nrow(rf_grid), "\n")
```

```
## Combinations: 27
```

5.3

```r
rf_wf <- workflow() |>
  add_recipe(recipe(class ~ ., data = train_data)) |>
  add_model(rf_tunable)

tune_results <- rf_wf |>
  tune_grid(resamples = cv_folds, grid = rf_grid,
            metrics = metric_set(accuracy, f_meas, roc_auc))
```

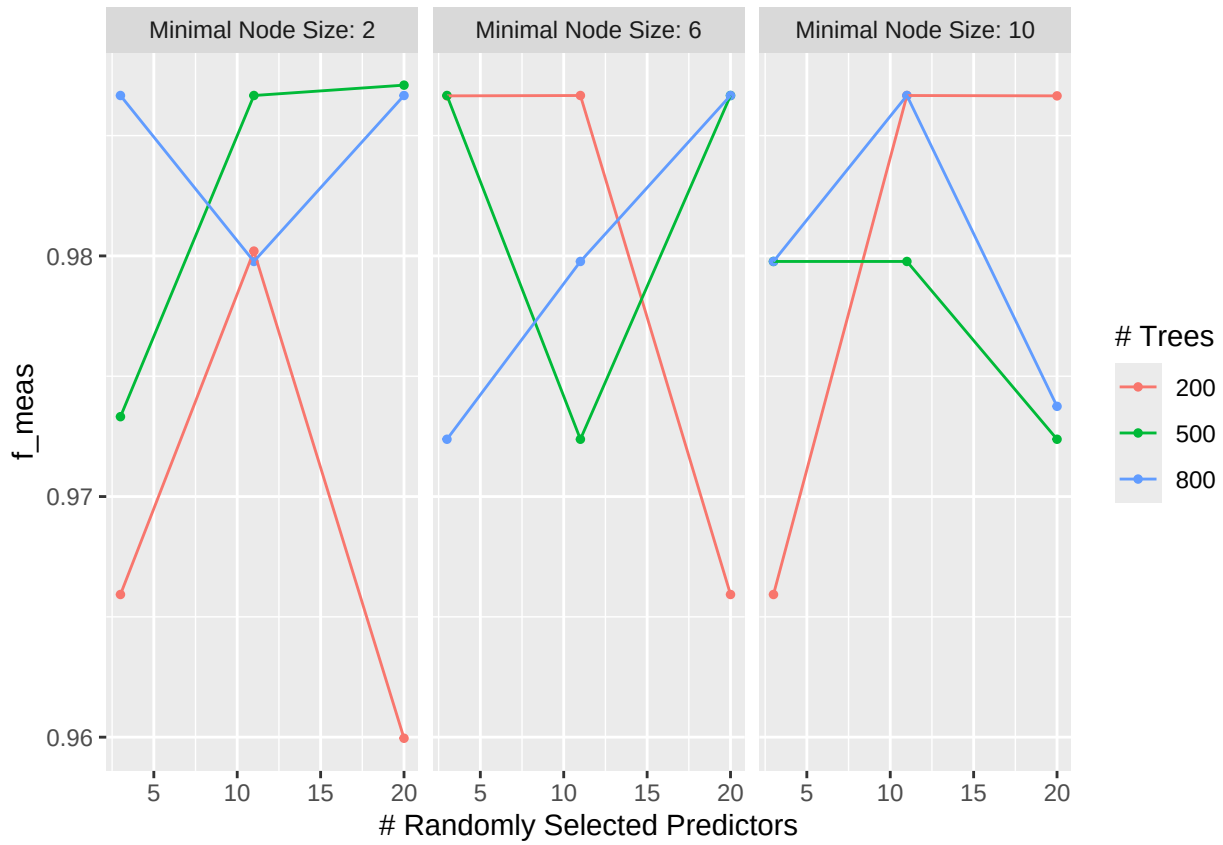
5.4

```r
tune_results |> show_best(metric = "f_meas", n = 5)
```

```
## # A tibble: 5 x 9
##    mtry trees min_n .metric .estimator  mean     n std_err .config
##   <int> <int> <int> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1    20   500     2 f_meas  binary     0.987     5 0.00790 Preprocessor1_Model06
## 2    11   500     2 f_meas  binary     0.987     5 0.0133  Preprocessor1_Model05
## 3     3   800     2 f_meas  binary     0.987     5 0.0133  Preprocessor1_Model07
## 4    20   800     2 f_meas  binary     0.987     5 0.0133  Preprocessor1_Model09
## 5    11   200     6 f_meas  binary     0.987     5 0.0133  Preprocessor1_Model11
```

5.5

```r
autoplot(tune_results, metric = "f_meas")
```



Q: Which hyperparameter seems to matter most? Does performance plateau after a certain number of trees?

From the tuning results (use `autoplot(tune_results)` to visualize), students should observe:

`mtry` (number of features considered at each split) is typically the most influential hyperparameter. Setting it too low means trees cannot access the best features at each split; setting it too high reduces the diversity between trees (defeating the purpose of the "random" in random forest). For classification, the default of square-root(p) (about 7 for our 50 genes) is usually a good starting point.

`min_n` (minimum node size) has a moderate effect. Very small values (1–2) allow trees to grow deep and potentially overfit; very large values (20+) create shallow trees that underfit. The effect is often less dramatic than `mtry`.

`trees` (number of trees) typically shows a clear **plateau**. Performance improves as you go from a small number (50–100) to a moderate number (300–500), but adding more trees beyond that yields diminishing returns. Going from 500 to 2000 trees rarely changes accuracy meaningfully — it just makes training slower.

This plateau happens because the ensemble's averaging effect stabilizes: once you have enough diverse trees, adding more does not provide new information. Unlike `mtry` and `min_n`, you generally cannot overfit by increasing the number of trees (more trees just converges to a stable estimate), so the only cost is computation time.

Practical takeaway: Focus your tuning budget on `mtry` and `min_n`. For `trees`, pick a value where performance has plateaued (typically 500–1000) and leave it fixed.

5.6

```r
best_params <- tune_results |> select_best(metric = "f_meas")
best_params
```

```
## # A tibble: 1 x 4
##    mtry trees min_n .config
##   <int> <int> <int> <chr>
## 1    20   500     2 Preprocessor1_Model06
```

```r
final_wf <- rf_wf |> finalize_workflow(best_params)
```

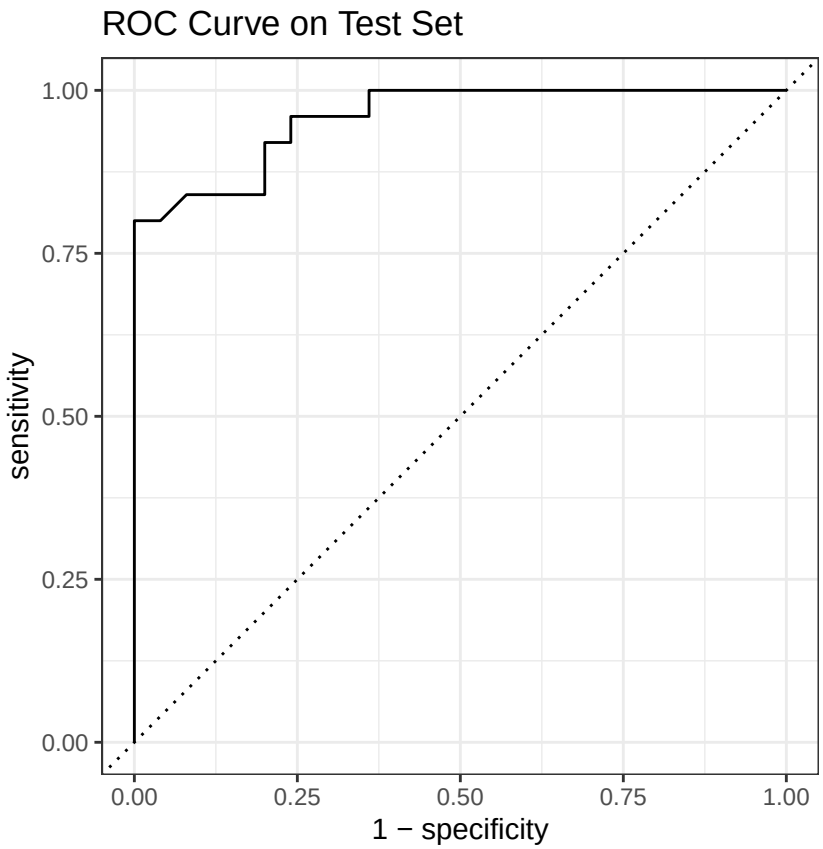
5.7

```r
final_fit <- final_wf |> last_fit(split = split)
final_fit |> collect_metrics()
```

```
## # A tibble: 3 x 4
##   .metric     .estimator .estimate .config
##   <chr>       <chr>          <dbl> <chr>
## 1 accuracy    binary        0.86   Preprocessor1_Model1
## 2 roc_auc     binary        0.958  Preprocessor1_Model1
## 3 brier_class binary        0.0956 Preprocessor1_Model1
```

5.8

```r
test_predictions <- final_fit |> collect_predictions()
test_predictions |>
  roc_curve(truth = class, .pred_normal) |>
  autoplot() + labs(title = "ROC Curve on Test Set")
```



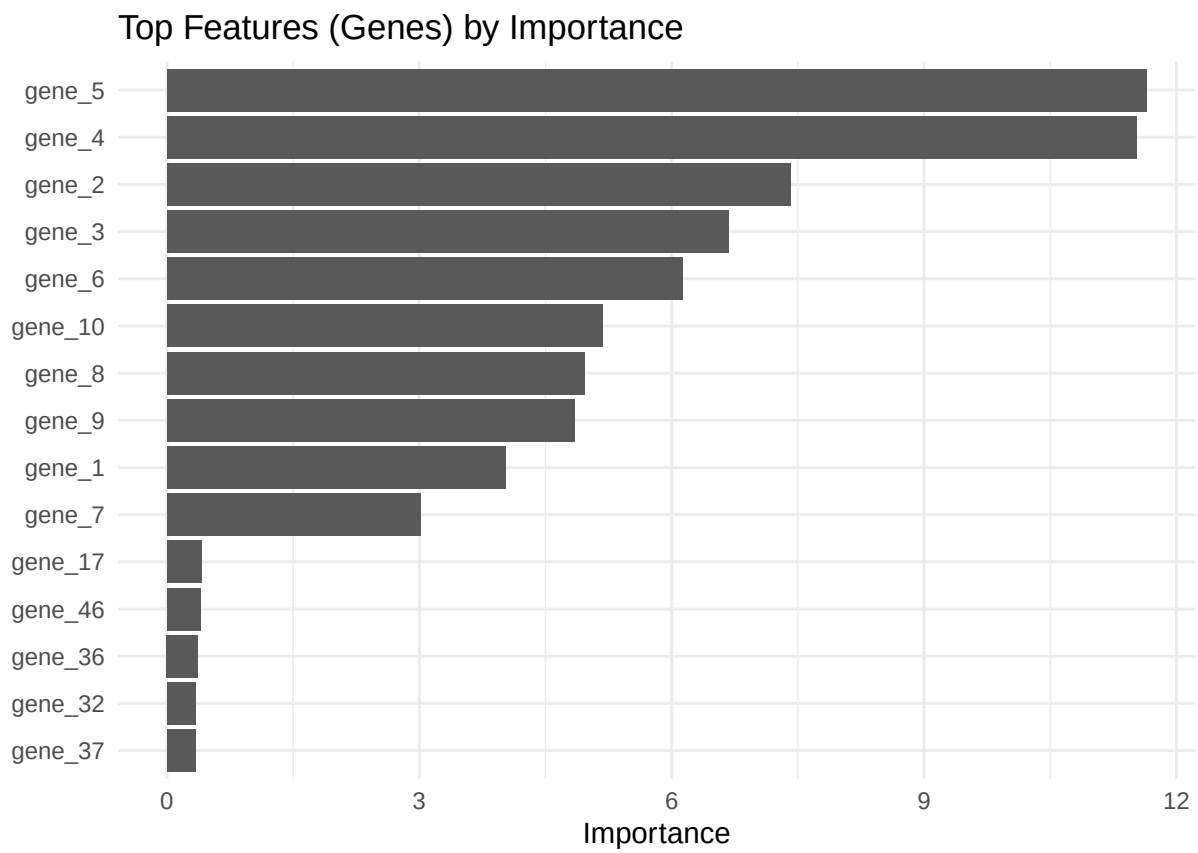
5.9

```
test_predictions |> conf_mat(truth = class, estimate = .pred_class)
```

```
##            Truth
## Prediction normal tumor
##     normal     23     5
##     tumor       2    20
```

5.10

```
final_model <- final_fit |> extract_fit_parsnip()
final_model |> vip(num_features = 15) +
  theme_minimal() + labs(title = "Top Features (Genes) by Importance")
```



Discussion Questions — Solutions

Q1: Do the most important genes match the truly DE genes (gene_1 to gene_10)? What does it mean if a noise gene appears as important?

Ideally the top features from the importance plot should be enriched for **gene_1 through gene_10**. In practice students will likely see that most of the top 10–15 features are indeed from the truly DE set, but one or two noise genes (e.g., gene_35 or gene_42) may also appear.

This happens because of **random correlations** — by chance, a noise gene may correlate with the outcome in the training data. This is not a bug; it is a feature of the method and an important lesson:

- Feature importance rankings are **estimates**, not ground truth.
- They should be interpreted as **hypothesis-generating**, not definitive.
- In a real experiment you would validate top features using independent data, literature evidence, or functional experiments.
- With small datasets, the importance of noise genes increases — another reason why larger sample sizes and repeated cross-validation improve reliability.

Q2: We used `importance = "impurity"` (Gini importance). The handout describes permutation importance. What is the difference?

	Gini importance (`"impurity"`)	Permutation importance (`"permutation"`)
How it works	Total reduction in Gini impurity across all splits in all trees where the feature is used	After the forest is built, randomly shuffle one feature at a time and measure the drop in accuracy
Speed	Fast — computed as a byproduct of tree building	Slower — requires additional model evaluations
Bias	Biased toward features with many unique values (high cardinality). A continuous gene with many split points tends to look more important than a binary feature, even if the binary feature is more predictive.	Unbiased and generally more reliable
Recommendation	Fine for quick exploration	Preferred for publication-quality results

Both are covered in the course slides (Random Forest — Important Concepts, slide 63).

Q3: In a real experiment, you would not know which genes are truly informative. How could you use feature importance to generate biological hypotheses?

Feature importance serves as a **hypothesis generation tool**, not as proof. A practical workflow:

1. Train your model and extract the top N most important genes.
2. Look up these genes in biological databases (e.g., GeneCards, KEGG, Gene Ontology) to check for involvement in relevant pathways.
3. Search the literature for known associations with your phenotype.
4. Genes that are both statistically important *and* biologically plausible become candidates for **experimental validation** (e.g., qPCR, knockdown, independent cohort replication).

This is where ML connects to biology: the model tells you *which features matter*, and your domain knowledge tells you *whether that makes biological sense*. A surprising gene in the top list could be a novel finding — or a false positive. That is why independent validation is essential.

Q4: We tuned 27 combinations. Would 1000 always give a better model? What are the risks?

Not necessarily. More combinations are **not always better** for several reasons:

1. **Computational cost:** 1000 combinations $\times$ 10 CV folds $\times$ 5 repeats = 50,000 model fits. With large datasets or complex models this can take hours or days.

2. **Overfitting to the validation set:** The more combinations you try, the higher the chance that the "best" one looks good by chance on your particular CV splits, not because it generalizes well. This is a subtle form of overfitting at the tuning level. Nested cross-validation (an outer CV around the entire tuning process) protects against this.

3. **Diminishing returns:** Model performance often plateaus after a modest search. The jump from 27 to 100 combinations may help; the jump from 100 to 1000 rarely makes a meaningful difference for random forests. The hyperparameter space is usually smooth enough that a coarse grid finds a near-optimal region.

4. **Random search as an alternative:** Instead of a regular grid with 1000 points, use `grid_random(size = 50)` — research has shown that random search finds good hyperparameters with far fewer evaluations than grid search, because it explores the space more efficiently and avoids wasting time on unimportant dimensions.

Practical recommendation: Start with a coarse grid (like our 27), examine the results, and only refine in the promising region if performance seems sensitive to the hyperparameters.